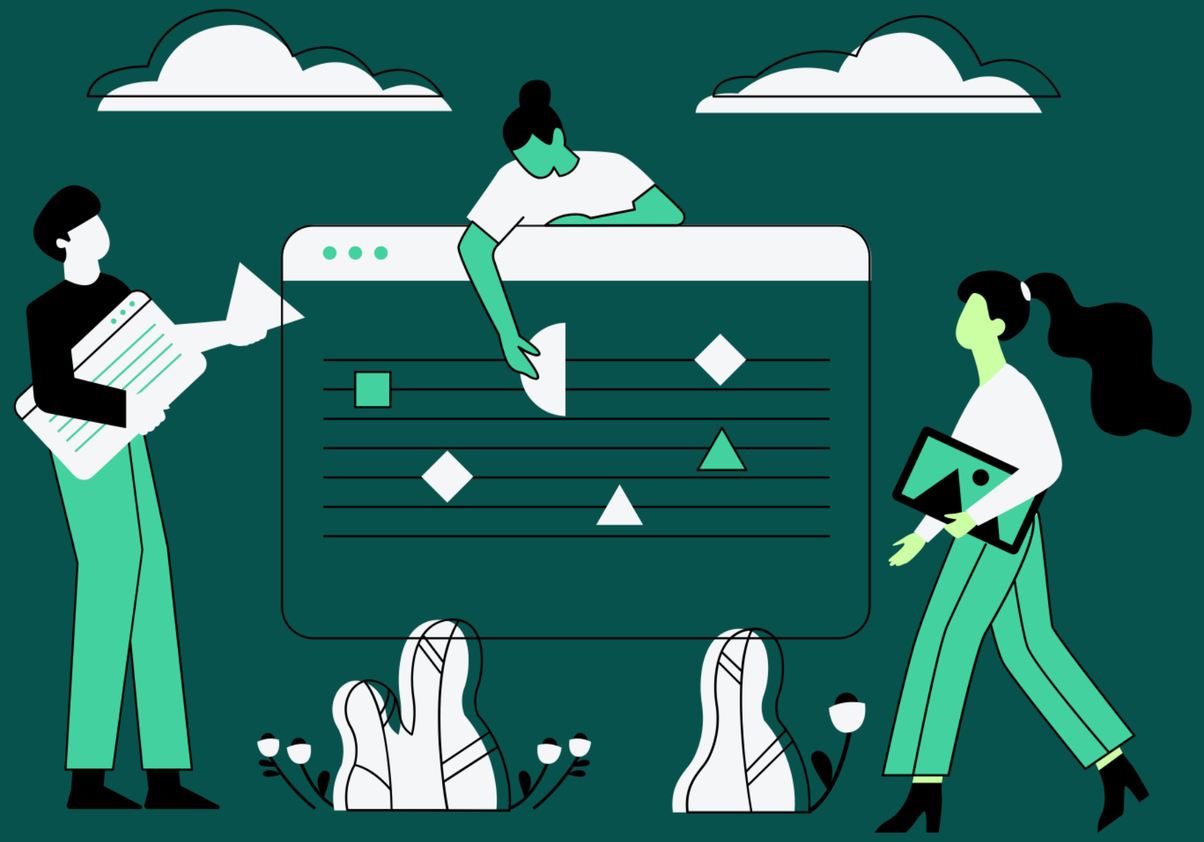
BLACK BOX TESTING

Group Members

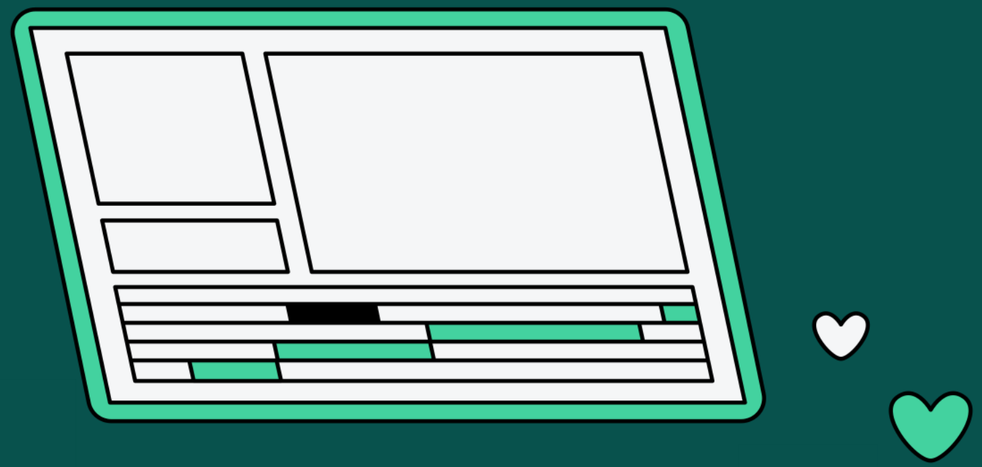


01 Ehtasham-ul-Haq

02 Abdullah

03 Hassan Ali

AGENDA

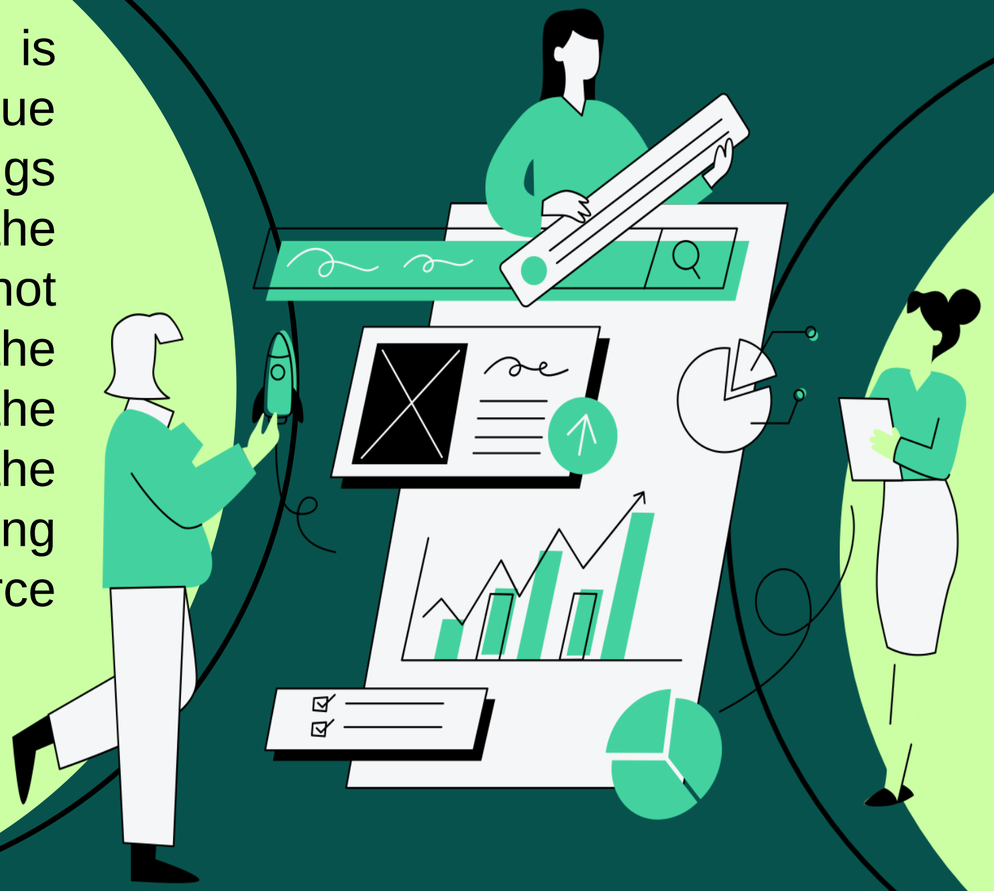


- Intro To Black Box Testing
- Types of Black Box Testing
- Techniques of Black Box Testing
- Example Template for Testing
- Features and Limitations
- Tools And Frameworks

INTRODUCTION

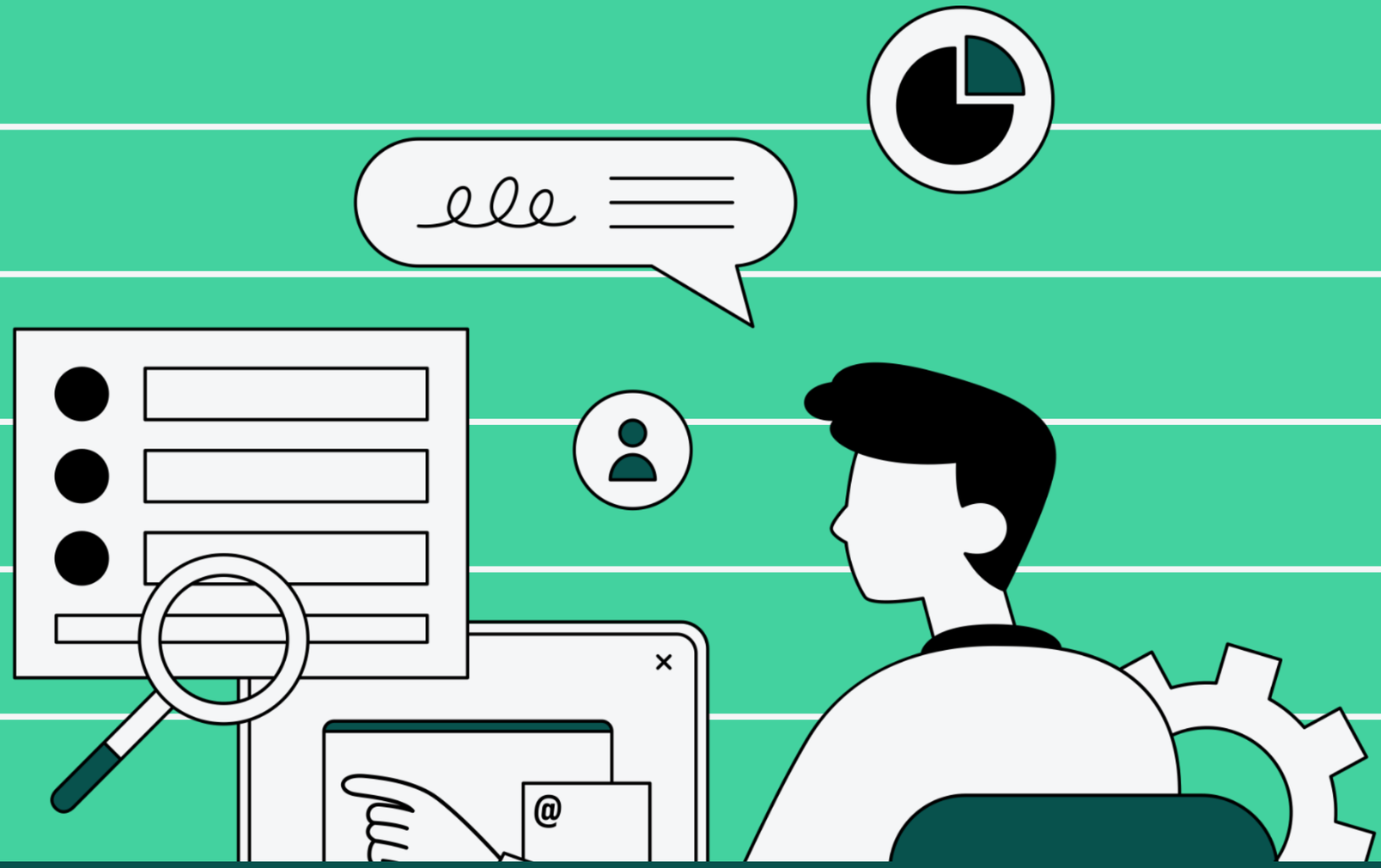
Introduction to Black-Box Testing

Black box testing is software testing technique where the internal workings or code structure of the system being tested are not known to the tester the tester focuses solely on the external behavior of the software, without having access to its internal source code.



Characteristics

- Independent Testing
- Requirements Based Testing
- Functional Testing
- No knowledge of Internal code



TYPES

Types of Black Box Testing

Functional Testing

Non Functional Testing

Regression Testing

User Interface (UI) Testing

Usability Testing

Ad Hoc Testing

Compatibility Issue

Security Testing

System Testing

Acceptance Testing

Testing Types Continue...

01

Functional Testing

It verifies that the software's functions and features work as expected and adhere to the specified requirements.
Example : Imagine a Music Player App (Spotify)
Playing a song works (search song by name , play song , listen to song) or Creating playlist saves the song into playlist . If you play the song and no sound comes it a functional defect

02

Non Functional Testing

It includes tests for performance, usability, security, scalability, reliability, and other quality attributes.
Example : For the same music App
Does the app load under 2 seconds even with slow internet connection ?

03

Regression Testing

It is performed to ensure that recent changes or updates to the software do not adversely affect existing functionality.

Example : Imagine a Music Player App (Spotify)

Suppose the music app adds a new “offline mode” feature. You re-test old features like playing a song or creating a playlist to ensure they still work. If the update causes playlists to delete randomly, that’s a regression defect.

04

User Interface Testing

It Tests the software’s graphical interface to ensure it’s visually correct, consistent, and easy to navigate.

Example : Consider a Food Delivery App (Food Panda)

The order now button is visible and clickable ? The menu layout is the same on iPhone and Android (e.g., no buttons are cut off).

05

Usability Testing

Tests how user-friendly and intuitive the software is for end-users, focusing on the overall experience.

Example : Your friends test the app by ordering something

either they struggle to find the payment option or find it confusing ?

06

Ad-Hoc Testing

Ad-hoc testing is an informal and unstructured testing approach where testers explore the software freely, executing test scenarios based on their intuition and experience.

Example : Imagine testing a video game like Candy Crush. You play it randomly—clicking buttons, skipping levels, or entering weird inputs (e.g., typing letters in a score field). If the game crashes when you type “abc” in the score, that’s an ad hoc discovery.

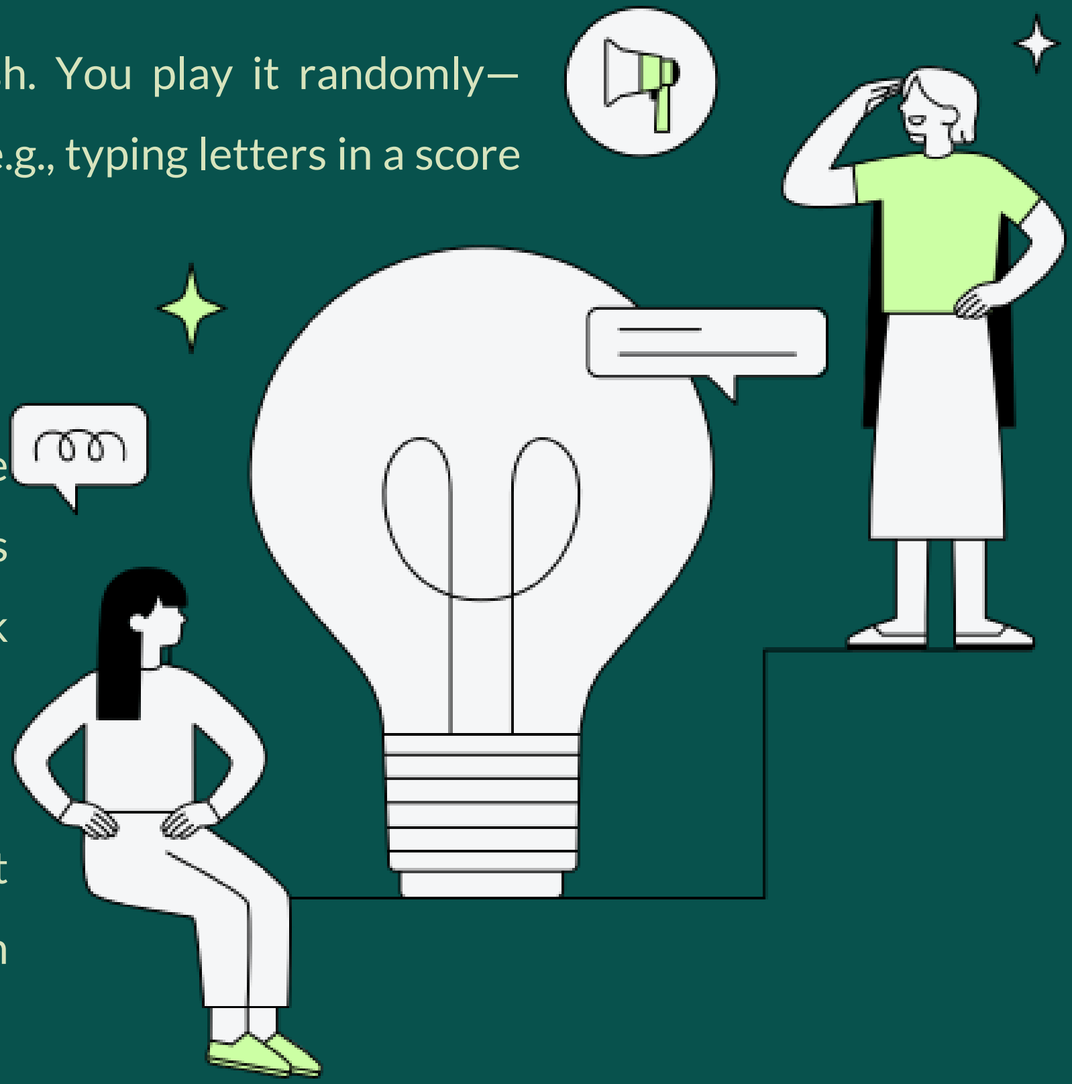
07

Compatibility Testing

Compatibility testing assesses how well the software performs across different environments, such as various browsers, operating systems, devices, and network configurations.

Example : Amazon (Shopping app)

It loads on an iPhone, Android, and a tablet, The checkout button works in Chrome and Safari. If the app crashes on Android but works on iPhone, that’s a compatibility issue.



08

Security Testing

Security testing aims to identify vulnerabilities and weaknesses in the software's security measures.
Example : Testing a Banking App
Entering a wrong password 10 times to see if it locks you out or Guessing someone else's login details. If you can log in with a fake password, that's a security defect.

09

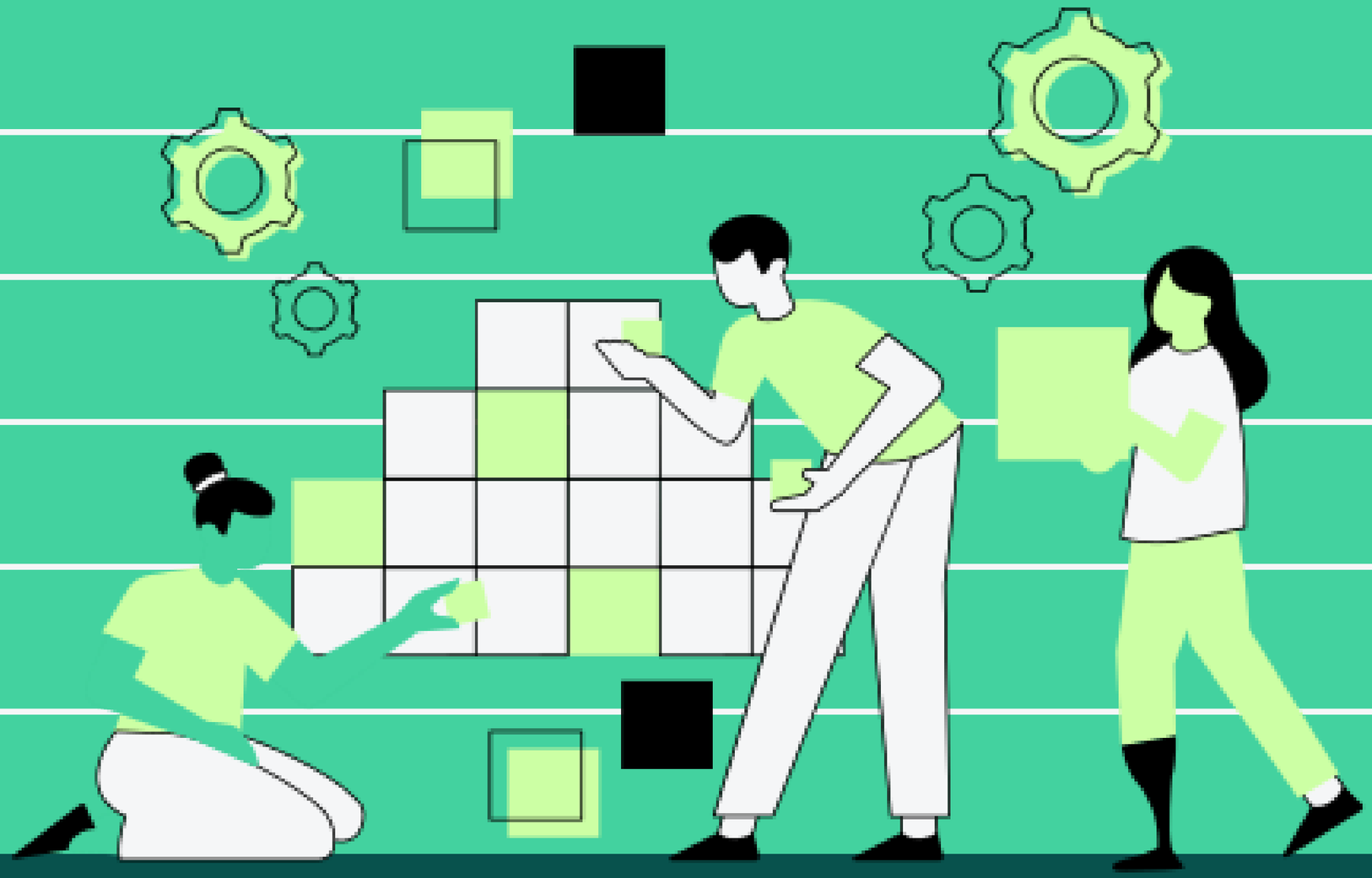
System Testing

Tests the entire software system as a whole to verify it meets all requirements.
Example : Netflix (Movie streaming App)
Log in, search for a movie, play it, and add it to favorites. If the movie doesn't play or the app crashes during playback, that's a system defect.

10

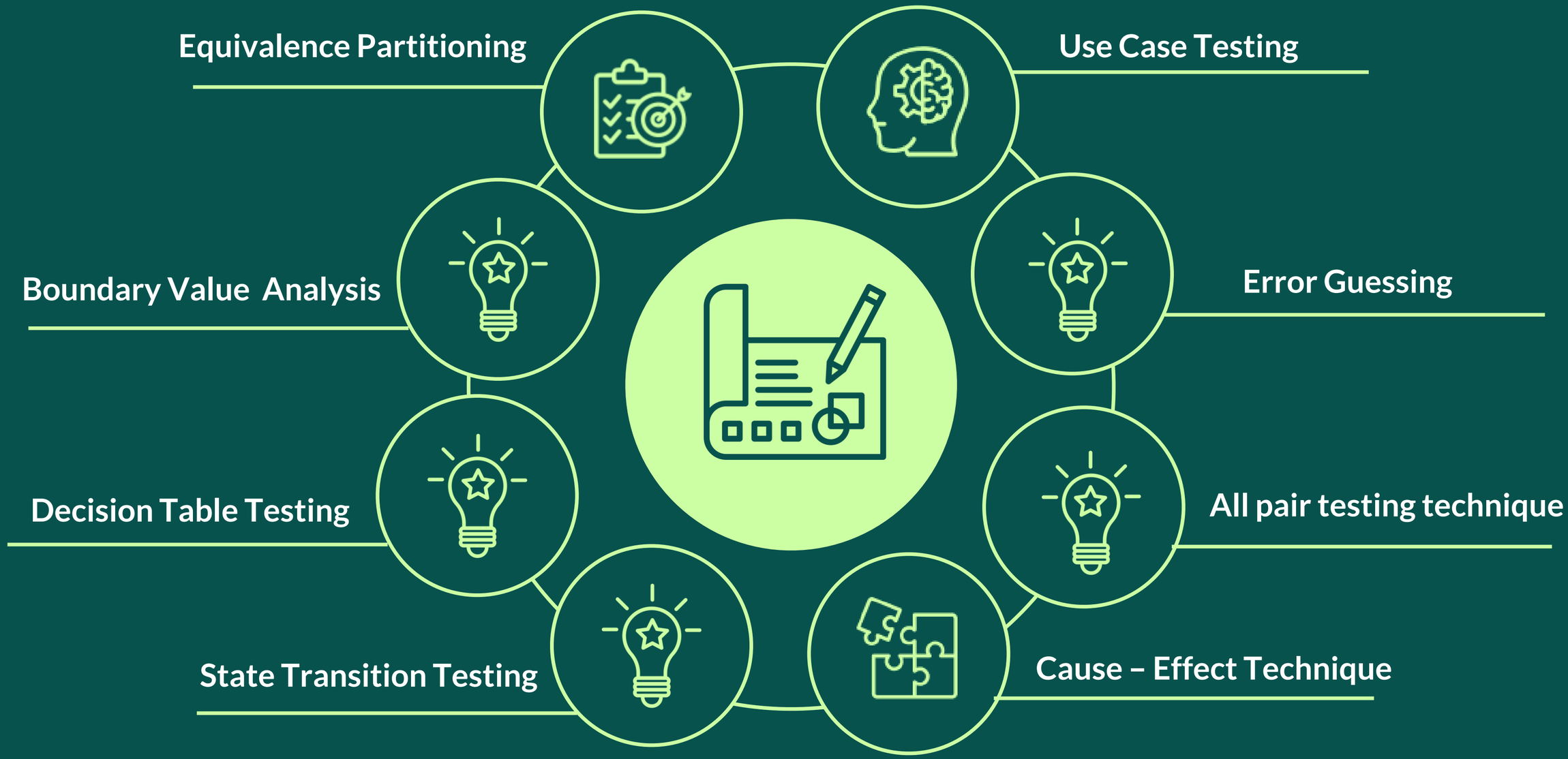
Acceptance Testing

Verifies if the software meets user needs and is ready for deployment. It's often the final testing phase.
Example : You give the movie streaming app to a group of friends to test. They check if they can find and watch their favorite movie. The interface is enjoyable. If they say it's too slow or missing features, it fails acceptance.



TECHNIQUES

Techniques of Black Box Testing



Techniques Continue...

01 Equivalence Partitioning

It Divides the input data into equivalent partitions, with each partition being regarded the same by the program. Testing one representative from each partition is usually enough to cover all potential scenarios.

Example : For a form that accepts age input between 18 and 65, equivalence partitions might include:

Valid partition: 18-65 (e.g., age 25), Invalid partition: Below 18 (e.g., age 15), Invalid partition: Above 65 (e.g., age 70)

02 Boundary Value Analysis

Tests the bounds of input ranges, as errors frequently arise on the edge of input limits.

Example : For an input field that accepts values from 1 to 100, boundary values would include:

Lower boundary: 1 Just below lower boundary: 0

Just above upper boundary: 101 Upper boundary: 100

03

Decision Table Testing

A decision table is used to represent and test different combinations of inputs and predicted outcomes. This method is effective for testing systems that involve several conditions and actions.

Example : For a loan application system with conditions like credit score (high/low) and income (above/below threshold), a decision table might include :

Credit Score	Income	Loan Approved
High	Above Threshold	Yes
High	Below Threshold	Yes
Low	Above Threshold	No
Low	Below Threshold	No

04 State Transition Testing

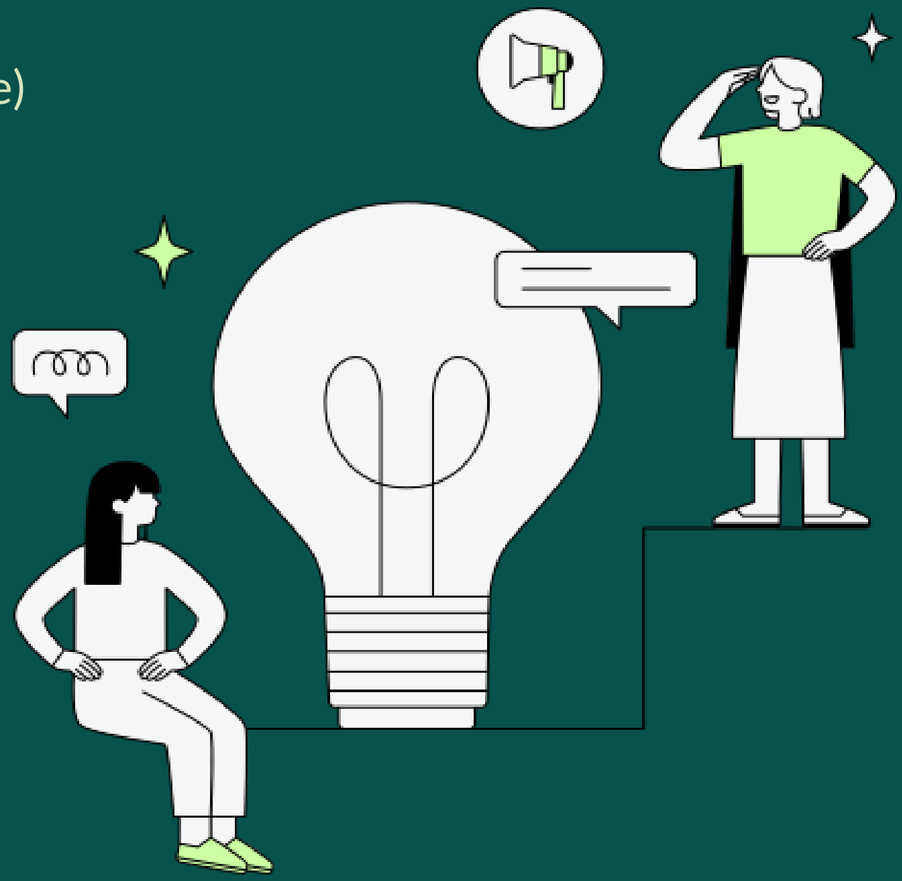
Tests the system's behavior in various states and transitions between them. It ensures that the system functions properly when transitioning from one state to another.

Example : For a user login system, states might include:

Logged Out Logged In Suspended

Transitions would be:

- From Logged Out to Logged In (successful login)
- From Logged In to Suspended (suspend account)
- From Suspended to Logged Out (logout from suspended state)



05 Use Case Testing

Focuses on validating the functionality of the system based on user interactions described in use cases. It ensures that the system meets the requirements of each use case.

Example : For an online shopping application, a use case might be:

- **Use Case:** Purchase Item
- **Steps:** Select item, add to cart, proceed to checkout, enter payment details, confirm purchase
- **Expected Outcome:** Order confirmation is displayed, and order is recorded

06 Error Guessing

Relies on the tester's experience and intuition to guess where errors might occur based on common mistakes, past experiences, and known problem areas.

Example : For a file upload feature, error guessing might include testing with

- ❑ Files of various types (e.g., .exe, .jpg, .pdf)
- ❑ Files with very large sizes
- ❑ Files with invalid extensions

07 All-Pair Testing

All-pair testing, also known as pairwise testing, is a combinatorial testing technique that aims to cover all possible pairs of input parameters in a test set.

Example : Consider a web application with three input parameters:

Parameter 1: Browser Type (Chrome, Firefox)

Parameter 2: Operating System (Windows, macOS)

Parameter 3: User Role (Admin, Guest)

Possible Test Cases

Browser: Chrome, OS: Windows, Role: Admin

Browser: Firefox, OS: Windows, Role: Guest

Browser: Chrome, OS: macOS, Role: Guest

Browser: Firefox, OS: macOS, Role: Admin

These test cases ensure that each pair of input values is tested, such as:

- ❑ Browser Type and Operating System
- ❑ Browser Type and User Role
- ❑ Operating System and User Role

08

Cause Effect Technique

The Cause-Effect approach, also known as Cause-Effect Graphing, is a black-box testing method that creates test cases based on the relationships between causes (inputs) and effects (outputs).

Key Concepts: Cause-Effect Graphing Cause Effect

Example : Consider an online account login system with the following input conditions (causes) and expected outputs (effects)

❑ **Causes:**

Correct username Correct password
Incorrect username Incorrect password
Account locked

❑ **Effects:**

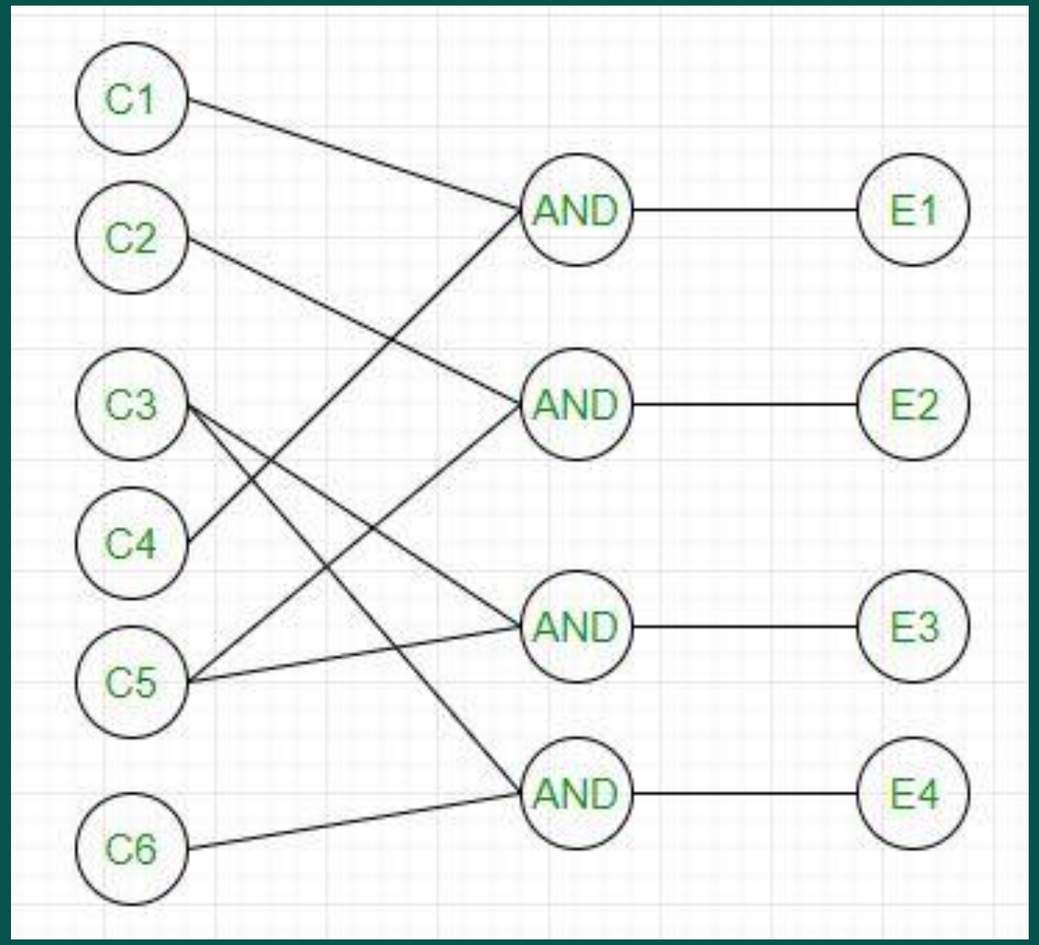
Login Success: If the username and password are both correct and the account is not locked.
Login Failure: If either the username or password is incorrect, or the account is locked.

❑ **Cause-Effect Graph:**

Cause 1 + Cause 2 → Effect 1 (Successful Login)
Cause 3 + Cause 4 → Effect 2 (Login Failure)
Cause 5 → Effect 3 (Account Locked)

❑ **Derived Test Cases:**

Correct username + Correct password (Expected: Login Success)
Correct username + Incorrect password (Expected: Login Failure)
Incorrect username + Correct password (Expected: Login Failure)
Account locked + Correct username + Correct password (Expected: Account Locked)



Cause Effect Graph

		1	2	3	4
CAUSES	C1	1	0	0	0
	C2	0	1	0	0
	C3	0	0	1	1
	C4	1	0	0	0
	C5	0	1	1	0
	C6	0	0	0	1
EFFECTS	E1	x	-	-	-
	E2	-	x	-	-
	E3	-	-	x	-
	E4	-	-	-	x

Graph Converted into Decision Table

EXAMPLE TEMPLATE:

Login functionality of a web application

Test Case Name: Verify successful login with valid credentials.
Test Steps:
(1) Open the web browse **(2)**Enter the URL of the application's login page **(3)**Enter valid username in the username field **(4)**Enter a valid password in the password field **(5)**Click on the "Login" button **(6)**Application process the login request.
Expected Result: The user should be successfully logged into the application's dashboard/homepage.
Test Case Status: PASS (if the user is redirected to the dashboard/homepage)
Test Case Name: Verify unsuccessful login with invalid credentials.
Test Steps:
(1) Open the web browser. **(2)**Enter the URL of the application's login page. **(3)**Enter an invalid username (e.g., "invalid user") in the username field. **(4)**Enter an invalid password (e.g., "wrong password") in the password field
(5) Click on the "Login" button. **(6)**Wait for the application to process the login request.

Expected Result: The login attempt should fail, and an appropriate error message (e.g., "Invalid username or password") should be displayed on the login page.
Test Case Status: PASS (if the error message is displayed)

FEATURES

Key Features:

01 Independence From Internal Implementation

02 User Centric Testing

03 Early detection of Interface Issues

04 Effective at Integration Testing

Effective for Requirement Validation

05

06

Suitable for Large Projects

LIMITATIONS

Limitations of Black Box Testing

TOOLS & FRAMEWORKS

TOOLS / FRAMEWORKS

Appium is an open-source test automation framework that allows testers to automate the testing of native, hybrid, and mobile web applications on both Android and iOS devices.

SoapUI is primarily known as an API testing tool, and its main focus is on testing the functionality and behavior of APIs (Web services).

Selenium

Load Runner

Appium

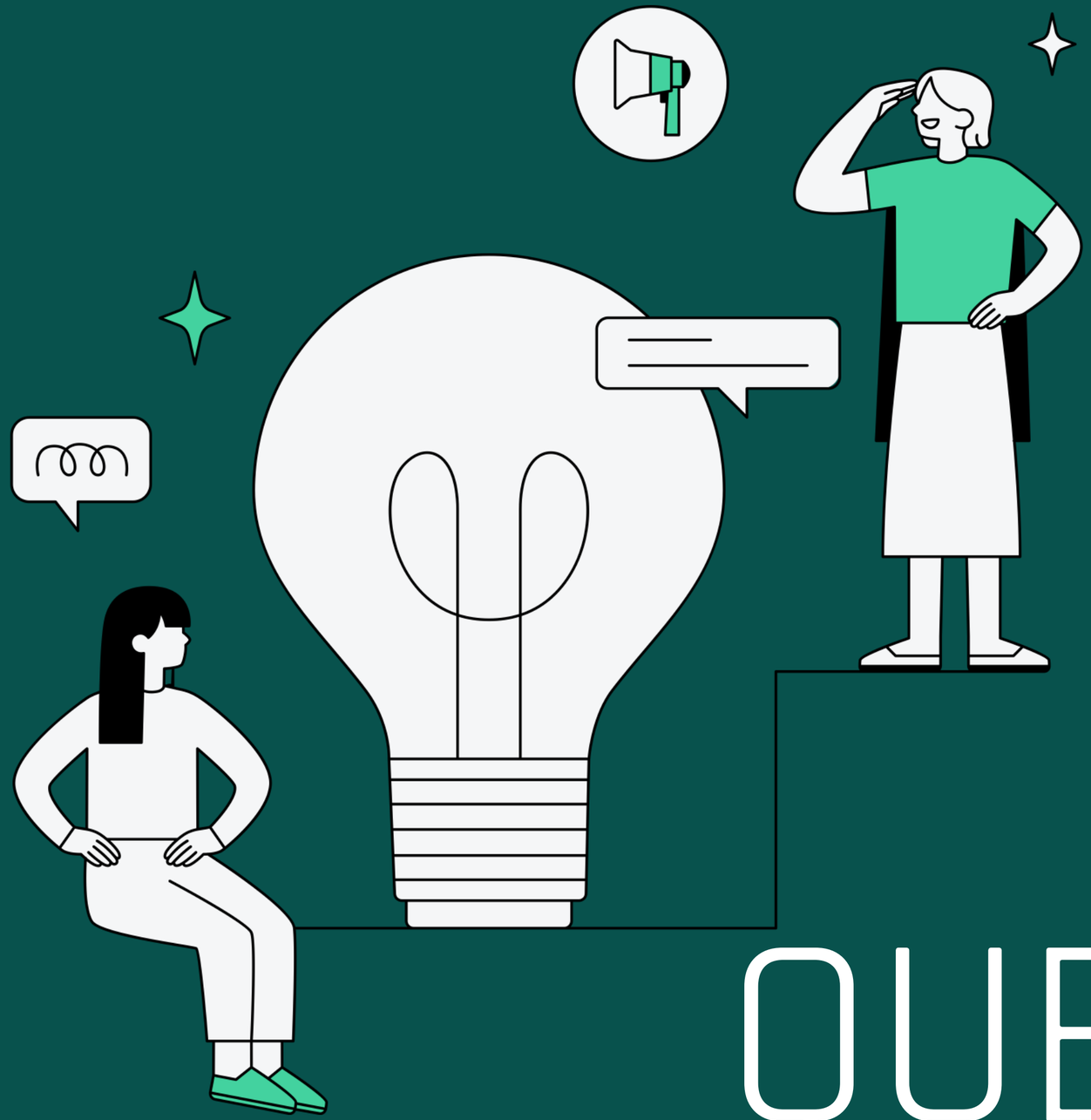
Soap UI

Selenium is an open-source testing framework that allows testers to automate the testing of web browsers, making it a valuable tool for performing black box testing on web-based systems.

LoadRunner is primarily known as a performance testing tool, and its core focus is on testing the performance, scalability, and reliability of applications under different load conditions.

References :

- What is Black Box Testing: Types, Tools & Examples (By Sourojit Das, Community Contributor - August 10, 2024) https://www.browserstack.com/guide/black-box-testing
- **Black Box Testing – Software Engineering** 09 Apr, 2025 https://www.geeksforgeeks.org/software-engineering-black-box-testing/
- **Black Box Testing** https://www.imperva.com/learn/application-security/black-box-testing/



THANK YOU

ANY QUESTIONS?